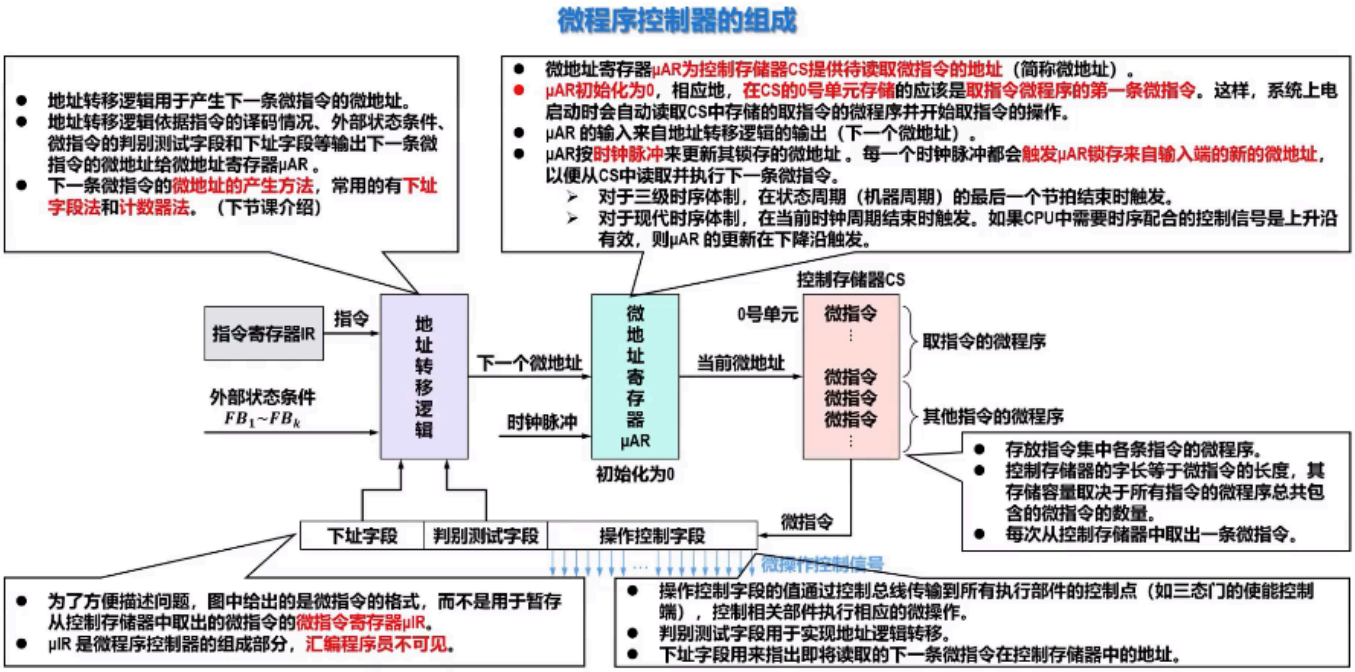


微程序控制器就是把“发控制信号”这件事，变成了一段一段存在存储器里的“程序”

一、背景回顾：控制器的作用

CPU 执行一条指令（比如 ADD），需要一堆部件配合：打开哪个寄存器、ALU 做什么运算、结果写到哪里……这些都靠控制器在正确的时刻发出正确的**控制信号**

实现控制器有两种思路：

- **硬布线**：用组合逻辑电路直接“搭”出控制信号，快但难改
- **微程序**（本图）：把每条机器指令的执行步骤写成一段“小程序”存进 ROM，执行指令就是“照着程序念步骤”。灵活、规整，是 CISC 机器的主流做法

关键类比：

微程序世界	生活类比
机器指令（如 ADD）	菜名“红烧肉”
微指令	菜谱里的一个步骤（“步骤3：焯水”）
微程序	一整道菜的完整做法
控制存储器 CS	装满所有菜谱的 cookbook（ROM）

二、四大部件

1 控制存储器 CS（粉色框）

一块 ROM，里面存放**所有微指令**：

- **0 号单元**：存的是“取指令的微程序”——不管执行什么指令，第一步都得先取指，所以这段是所有指令**共享的开场白**
- **后面的单元**：每条机器指令对应自己的一段微程序（其他指令的微程序）

2️⃣ 微地址寄存器 μAR（青色框）

存的是"**当前正在执行的微指令在 CS 中的地址**"
- 它的作用就像主存层面的 PC（程序计数器），只不过管的是**微程序这一层**；
- **初始化为 0** 非常巧妙：机器一上电，μAR = 0，自动指向 CS 的 0 号单元 → 先执行取指微程序 → 取出第一条机器指令。整个 CPU 就此启动

3️⃣ 微指令的三个字段（底部横条）

从 CS 读出一条微指令后，它被拆成三段，各司其职：

字段	作用	一句话理解
操作控制字段	向外发出一排**微操作控制信号**（图中向下的箭头）	"这一步该干活的干活的指令"，直接打开/关闭数据通路上各个门
下址字段	直接填好"下一条微指令在 CS 的地址"	步骤末尾写着"下一步见步骤 4"
判别测试字段	告诉地址转移逻辑："此处需要分支判断！"	步骤末尾写着"如果肉没烂，回到步骤 2"
⚠️ 注意图中的连线：下址字段和判别测试字段会**反馈回**地址转移逻辑，而操作控制字段是输出去控制数据通路的——它俩分工完全不同		
### 4️⃣ 地址转移逻辑（紫色框）		
这是一个**组合逻辑电路**，任务是回答："**下一条微指令的地址是多少？**"		
它综合四路信息做决策：		
1. **指令寄存器 IR** 送来的操作码——决定"该跳到哪条指令的微程序入口"（比如 ADD 的微程序和 SUB 的微程序开头不同）；		
2. **外部状态条件 FB_1~FB_k**——比如进位标志、零标志，用于条件分支；		
3. 微指令的**判别测试字段**——"本步要不要做判断"；		
4. 微指令的**下址字段**——默认的下一地址		

## 三、把整个流程串起来走一遍 🔄		

① 上电 → μAR = 0
② μAR 的内容（当前微地址）去 CS 寻址 → 读出 0 号单元的取指微指令
③ 操作控制字段发出信号 → 完成取指，机器指令进入 IR
④ 下址字段 + 判别测试字段 + IR 中的操作码 → 地址转移逻辑算出
该指令微程序入口地址 → 在时钟脉冲驱动下写入 μAR
⑤ 如此逐条执行微指令，每条都发一串控制信号
⑥ 一条机器指令执行完 → 地址转移逻辑让流程回到取指微程序
⑦ 循环往复……

时钟脉冲的作用：μAR 是时序部件，它在时钟边沿才更新，保证"一条微指令执行完，才切换到下一条"——这是整个微程序机的节拍器

四、一句话总结

> **微程序控制器 = 用"存储好的微指令 + 一个微型取指机构"代替复杂的控制信号发生电路。** CS 是"剧本库"，μAR 是"提词器上的指针"，地址转移逻辑是"导演"，决定下一幕演什么——而每一幕（微指令）都会向整个 CPU 打出一串灯号（微操作控制信号），指挥各部件协同演出一条机器指令的完整执行